These two functions are similar in effect to the previous two, but the desired escape characters must be specified using a second argument. Useful when preparing strings for use in weird and arbitrary formats, which is a fairly niche use-case. The list of characters can simply be all the bytes requiring escape or a range (specified by a special `..` sequence with the lowest value character going first).

```
$ship = 'What Are the Civilian Applications?';
print addcslashes($ship, 'Celrtu') . PHP_EOL;
print addcslashes($ship, 'e..r');
```

```
Output:
Wha\t A\r\e \th\e \Civi\lian App\lica\tions?
W\hat A\r\e t\h\e C\iv\i\l\ia\n A\p\p\l\icat\i\o\ns?
```

Although not shown here, it is possible for the second argument to hold multiple ranges, such as `"\x00..\x1F\x41..\x5A"` (which would put backslashes in front of every ASCII control character and every capital ASCII letter, not that I can think of a good reason to do so).

quotemeta()

Like the previous escape functions, `quotemeta()` will prepend a certain set of ASCII bytes with a backslash. This time, it's the regular expression meta-characters, which are `. \ + * ? [ ^ ] ( $ )`. We're going to dive headfirst into regular expressions (you'll either love them or hate them!) later in the chapter, so let's not waste time here dissecting what each of those symbols means. Here's a quick example anyway, even though it's likely you'll never need to use this function (there's a better alternative).

```
$pattern = 'What Are the Civilian Applications?';
$regex = '/^' . quotemeta($pattern) . '$/';
print $regex;
```

```
Output:
/^What Are the Civilian Applications\?$/
```

Note that the question mark character in `$ship` is now escaped (because `?` has special effects in these expressions, you'll see).